

Lambda

Why Lambda?

- A “lambda expression” is a block of code that you can pass around so it can be executed later, once or multiple times.

Event Handling

```
btn.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        System.out.println("OK");  
    }  
});
```

- Code for data — The object passed to the btn is code.

```
btn.addActionListener(event -> System.out.println("OK"));
```

Lambda Expression

```
Runnable run = ()->System.out.println("running");  
new Thread(run).start();
```

```
Runnable run2 = ()->{  
    System.out.println("running");  
    System.out.println("end");  
};  
new Thread(run2).start();
```

Lambda Expression

```
BinaryOperator<Long> add = (x,y) -> x+y;  
System.out.println(add.apply(100L, 200L));
```

```
BinaryOperator<Long> addExp = (Long x, Long y) -> x+y;  
System.out.println(addExp.apply(100L, 200L));
```

value vs variable

```
public class ButtonName extends JFrame {  
    String name;  
    JButton btn;  
  
    public String getName() { return name; }  
    public void setName(String nn) { name = nn; }  
  
    public ButtonName(String nn) {  
        name = nn;  
        String thisName = getName();  
        btn = new JButton("OK");  
        btn.addActionListener(event -> System.out.println(getName()));  
        add(btn);  
    }  
}
```

final

```
public ButtonName(String nn) {  
    name = nn;  
    String thisName = getName();  
    btn = new JButton("OK");  
    btn.addActionListener(event -> System.out.println(nn));  
    nn = "lala";  
    add(btn);  
}
```


Functional Interface

- Interface that has one function.

```
interface Func {  
    void ff();  
}
```

```
public class FuncInterface {  
    public void app(Func f) {  
        f.ff();  
    }  
}
```

```
    public static void main(String[] args) {  
        FuncInterface fi = new FuncInterface();  
        fi.app()->System.out.println("Hello");  
    }  
}
```


type inference

```
HashMap<String, String> h = new HashMap<>();
```

```
private void useHashMap(Map<String, String> v) {}  
useHashMap(new HashMap<>());
```

Predicate

```
Predicate<Integer> atLeast5 = x-> x>5;
```

```
public interface Predicate<T> {  
    boolean test(T t);  
}
```

BinaryOperator

```
BinaryOperator<Integer> addInt = (x,y) -> x+y;
```

Key Points

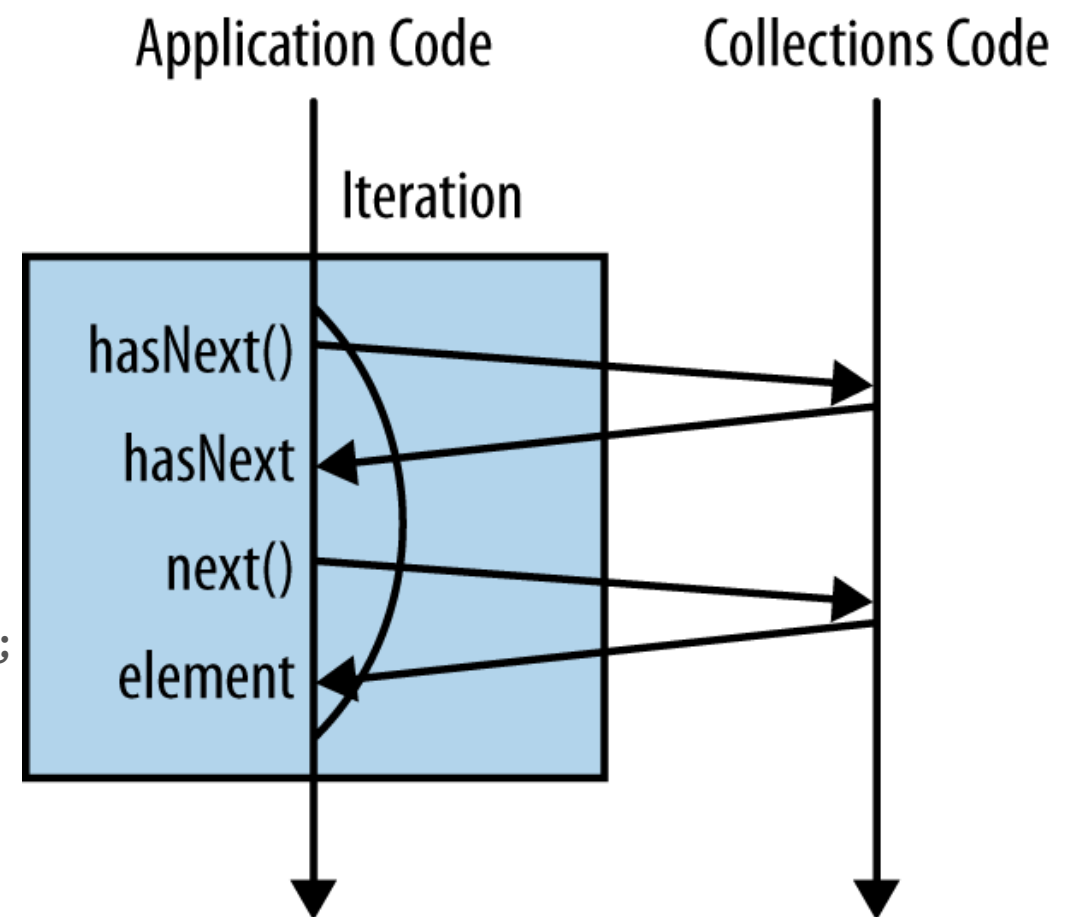
- A lambda expression is a method without a name that is used to pass around behavior as if it were data.
- Lambda expressions look like this:
`BinaryOperator<Integer> add = (x, y) -> x + y.`
- A functional interface is an interface with a single abstract method that is used as the type of a lambda expression.

Stream

External Iteration

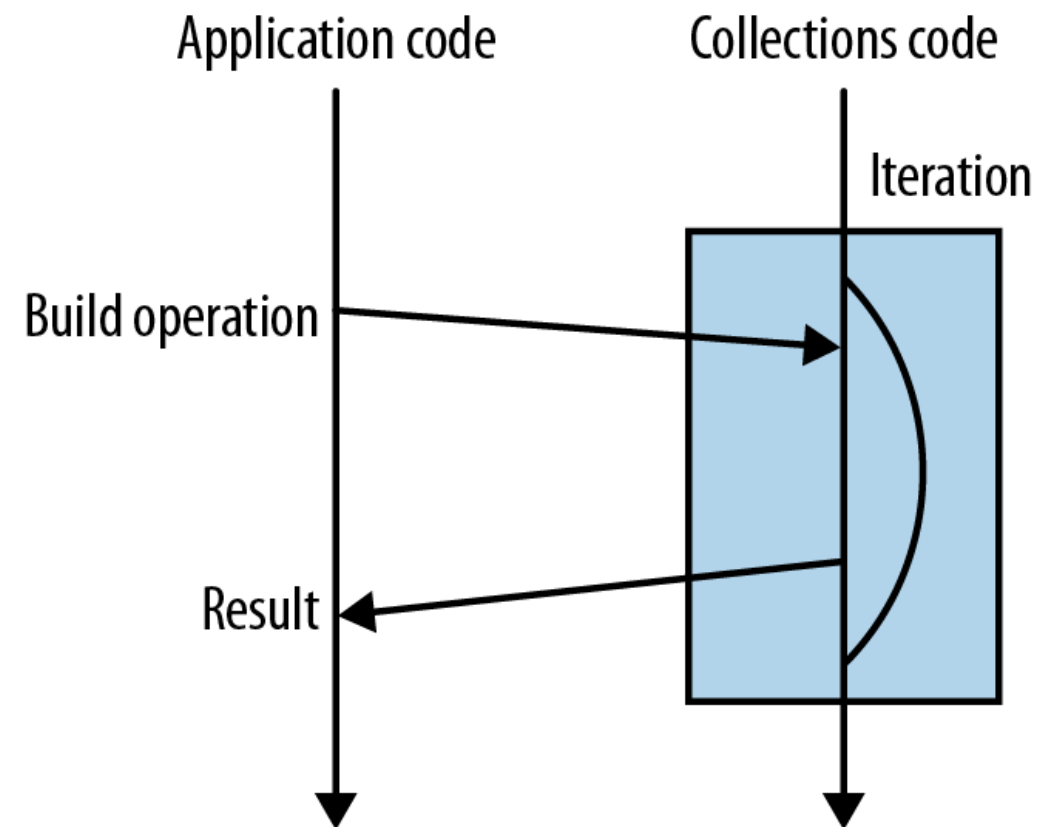
```
int count = 0;
for (Artist artist : allArtists) {
    if (artist.isFrom("London")) {
        count++;
    }
}
```

```
int count = 0;
Iterator<Artist> iterator = allArtists.iterator();
while(iterator.hasNext()) {
    Artist artist = iterator.next();
    if (artist.isFrom("London")) {
        count++;
    }
}
```



Internal Iteration

```
long count = allArtists.stream()  
    .filter(artist -> artist.isFrom("London"))  
    .count();
```



What's Actually Going On

```
allArtists.stream().filter(artist -> artist.isFrom("London"));
```

```
allArtists.stream()  
    .filter(artist -> {  
        System.out.println(artist.getName());  
        return artist.isFrom("London");  
    });
```

lazy way

lazy way

- `filter()` returns `Stream` —> lazy way
- `count()` returns `int` —> eager way

Common Stream Operations

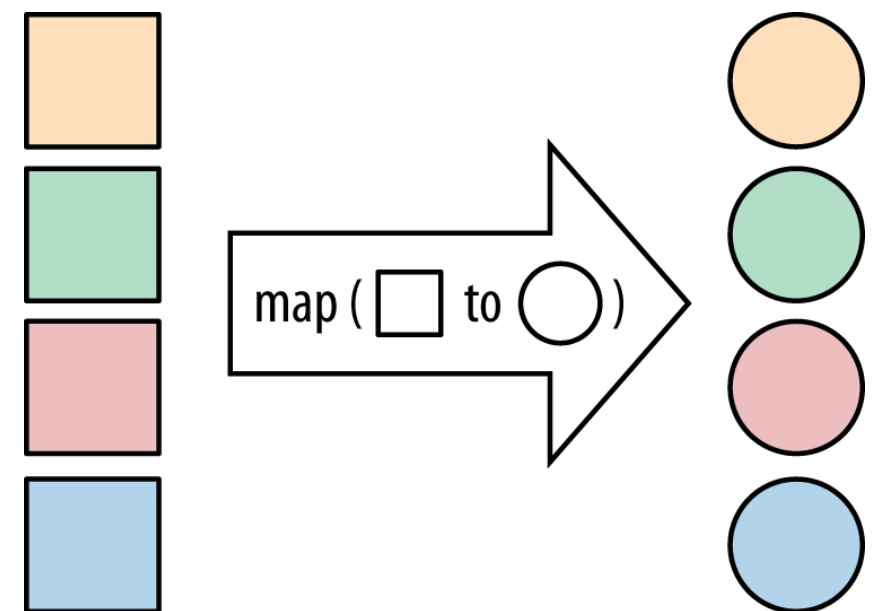
- `collect(toList())`
 - an eager operation that generates a list from the values in a Stream

```
List<String> collected = Stream.of("a", "b", "c").collect(Collectors.toList());  
assertEquals(Arrays.asList("a", "b", "c"), collected);
```

Common Stream Operations

- map
 - If you've got a function that converts a value of one type into another, map lets you apply this function to a stream of values, producing another stream of the new values.

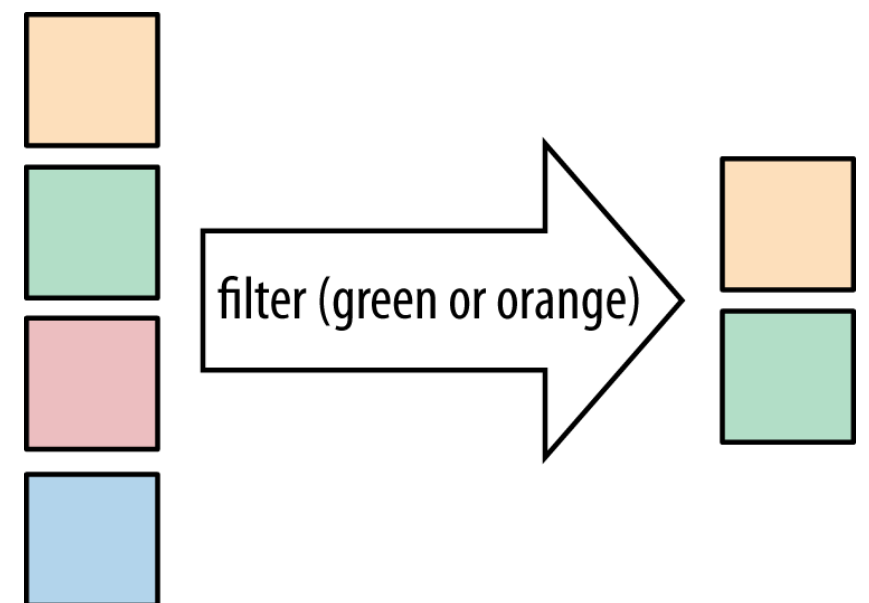
```
List<String> collected = Stream.of("a", "b", "hello")  
    .map(string -> string.toUpperCase())  
    .collect(toList());  
assertEquals(asList("A", "B", "HELLO"), collected);
```



Common Stream Operations

- filter
 - Any time you're looping over some data and checking each element, you might want to think about using the new filter method on Stream

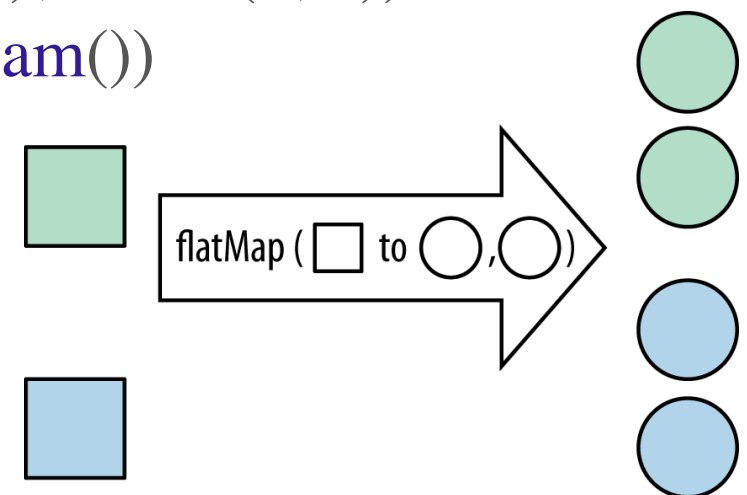
```
List<String> beginningWithNumbers  
= Stream.of("a", "1abc", "abc1")  
  .filter(value -> isDigit(value.charAt(0)))  
  .collect(toList());
```



Common Stream Operations

- flatMap
 - flatMap lets you replace a value with a Stream and concatenates all the streams together.

```
List<Integer> together = Stream.of(Arrays.asList(1, 2), asList(3, 4))  
    .flatMap(numbers -> numbers.stream())  
    .collect(toList());  
assertEquals(asList(1, 2, 3, 4), together);
```



Common Stream Operations

```
List<Track> tracks = asList(new Track("Bakai", 524),  
    new Track("Violets for Your Furs", 378),  
    new Track("Time Was", 451));  
  
Track shortestTrack = tracks.stream()  
    .min(Comparator.comparing(track -> track.getLength()))  
    .get();  
assertEquals(tracks.get(1), shortestTrack);
```

Common Pattern

```
Track shortestTrack = tracks.get(0);  
for (Track track : tracks) {  
    if (track.getLength() < shortestTrack.getLength()) {  
        shortestTrack = track;  
    }  
}
```

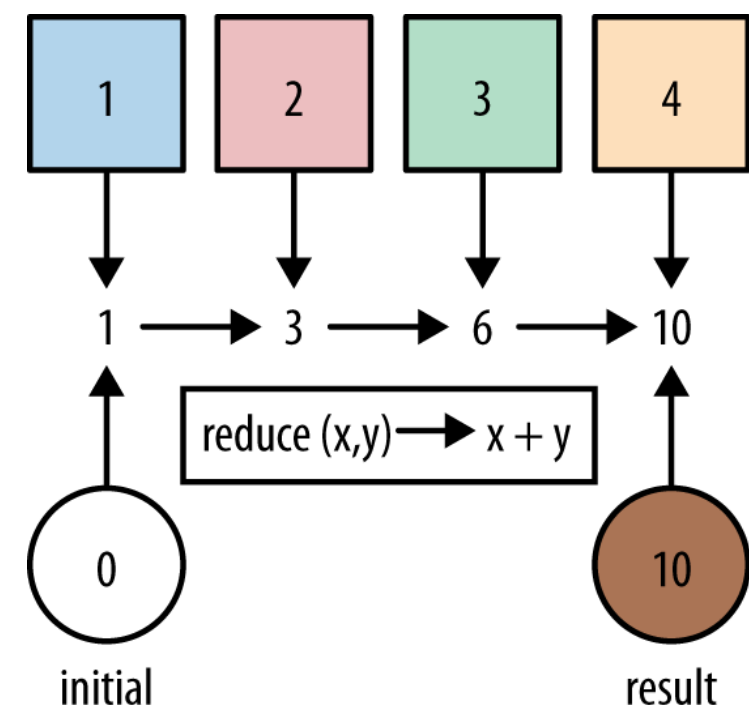
```
Object accumulator = initialValue;  
for(Object element : collection) {  
    accumulator = combine(accumulator, element);  
}
```

- The Reduce Pattern

Common Stream Operations

- reduce
 - Use the reduce operation when you've got a collection of values and you want to generate a single result.

```
int count = Stream.of(1, 2, 3)
    .reduce(0, (acc, element) -> acc + element);
```



Putting Operations Together

1. Get all the artists for an album.
2. Figure out which artists are bands.
3. Find the nationalities of each band.
4. Put together a set of these values.

solution

1. There's a nice `getMusicians` method on our `Album` class that returns a `Stream`.
2. We use `filter` to trim down the artists to include only bands.
3. We use `map` to turn the band into its nationality.
4. We use `collect(toList())` to put together a list of these nationalities.

```
Set<String> origins = album.getMusicians()  
    .filter(artist -> artist.getName().startsWith("The"))  
    .map(artist -> artist.getNationality())  
    .collect(toSet());
```

Multiple Stream Calls

- It's harder to read what's going on because the ratio of boilerplate code to actual business logic is worse.

```
List<Artist> musicians = album.getMusicians()  
    .collect(toList());
```

```
List<Artist> bands = musicians.stream()
```

- It's less efficient because it requires eagerly creating new collection objects at each intermediate step.

```
    .filter(artist -> artist.getName().startsWith("The"))  
    .collect(toList());
```

```
Set<String> origins = bands.stream()
```

```
    .map(artist -> artist.getNationality())
```

```
    .collect(toSet());
```

- It clutters your method with meaningless garbage variables that are needed only as intermediate results.

- Good

```
Set<String> origins = album.getMusicians()
```

- It makes operations harder to automatically parallelize.

```
    .filter(artist -> artist.getName().startsWith("The"))  
    .map(artist -> artist.getNationality())
```

```
    .collect(toSet());
```

Good Use of Lambda Expressions

- Functions with no side effects don't change the state of anything else in the program or the outside world.

```
private ActionEvent lastEvent; private void registerHandler() {  
    button.addActionListener((ActionEvent event) -> {  
        this.lastEvent = event; });  
}
```

```
ActionEvent localEvent = null; button.addActionListener(event -> {  
    localEvent = event;  
});
```

Key Points

- Internal iteration is a way of iterating over a collection that delegates more control over the iteration to the collection.
- A Stream is the internal iteration analogue of an Iterator.
- Many common operations on collections can be performed by combining methods on Stream with lambda expressions.

Libraries

Using Lambda Expressions

- `logger.log(DEBUG, "Look at this"+expensiveOp());`
- `logger.debug(() -> "Look at this: " + expensiveOperation());`
- ```
public void debug(Supplier<String> message) {
 if (isDebugEnabled()) {
 debug(message.get());
 }
}
```



# Primitives Issues

- Java generics are based around erasing a generic parameter—in other words, pretending it's an instance of Object—only the boxed types can be used as generic arguments.
- Because boxed types are objects, there is a memory overhead to them.
- There is also a computational overhead when converting from a primitive type to a boxed type, called boxing, and vice versa, called unboxing.



# Primitives Solution

- The streams library differentiates between the primitive and boxed versions of some library functions.
- Only the int, long, and double types have been chosen as the focus of the primitive specialization implementation in Java 8 because the impact is most noticeable in numerical algorithms.
- The primitive specializations have a very clear-cut naming convention. If the return type is a primitive, the interface is prefixed with To and the primitive type, as in ToLongFunction
- If the argument type is a primitive type, the name prefix is just the type name, as in LongFunction
- There are also specialized versions of Stream for these primitive types that prefix the type name, such as LongStream.

```
IntSummaryStatistics trackLengthStats = album.getTracks().mapToInt(track -> track.getLength()).summaryStatistics();
```

# Overload Resolution

- Function overloading poses a problem for parameter-type inference because it means that there are several types that could be inferred.

```
private void overloadedMethod(Object o) {
 System.out.print("Object");
}
```

```
private void overloadedMethod(String s) {
 System.out.print("String");
}
```

```
overloadedMethod("abc");
```

# Another Choice

```
private interface IntegerBiFunction extends BinaryOperator<Integer> {
}

private void overloadedMethod(BinaryOperator<Integer> lambda)
{ System.out.print("BinaryOperator");
}

private void overloadedMethod(IntegerBiFunction lambda)
{ System.out.print("IntegerBinaryOperator");
}

overloadedMethod((x, y) -> x + y);
```

# Ambiguous

```
overloadedMethod((x) -> true);
```

```
private interface IntPredicate { public boolean test(int value);
}
```

```
private void overloadedMethod(Predicate<Integer> predicate)
{ System.out.print("Predicate");
}
```

```
private void overloadedMethod(IntPredicate predicate)
{ System.out.print("IntPredicate");
}
```

# @FunctionalInterface

- is an annotation that should be applied to any interface that is intended to be used as a functional interface.
- Using the annotation compels javac to actually check whether the interface meets the criteria for being a functional interface. If the annotation is applied to an enum, class, or annotation, or if the type is an interface with more than one single abstract method, then javac will generate an error message.

# Binary Interface Compatibility

- Here the guarantee is that if you've got source code in Java 1-7, it'll compile in Java 8.
- The stream method was added to the Collection interface in Java 8, which means that any class that implements Collection must also have this method on it.
- It means that any class outside of the JDK that implements Collection—say, MyCustomList—must also have implemented the stream method.

# Default Methods

- The Java 8 approach to solving the problem is to allow Collection to say, “If any of my children don’t have a stream method, they can use this one.” These methods on an interface are called default methods. They can be used on any interface, functional or not.

```
default void forEach(Consumer<? super T> action) {
 for(Tt:this){
 action.accept(t);
 }
}
```



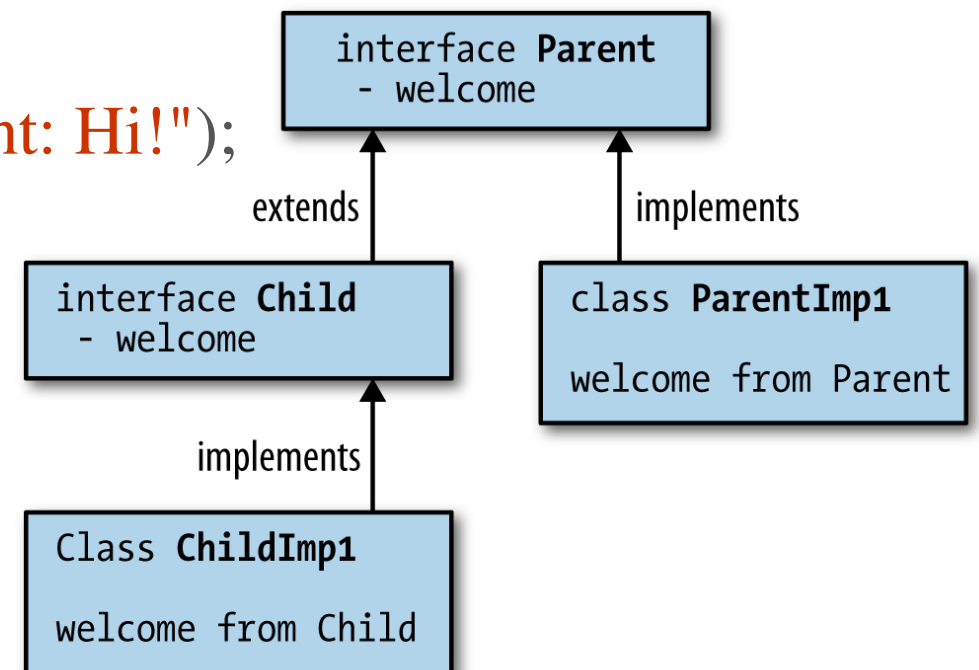
# Default Methods and Subclassing

```
public interface Parent {
 public void message(String body);
 public default void welcome() { message("Parent: Hi!"); }
}

public String getLastMessage(); }
```

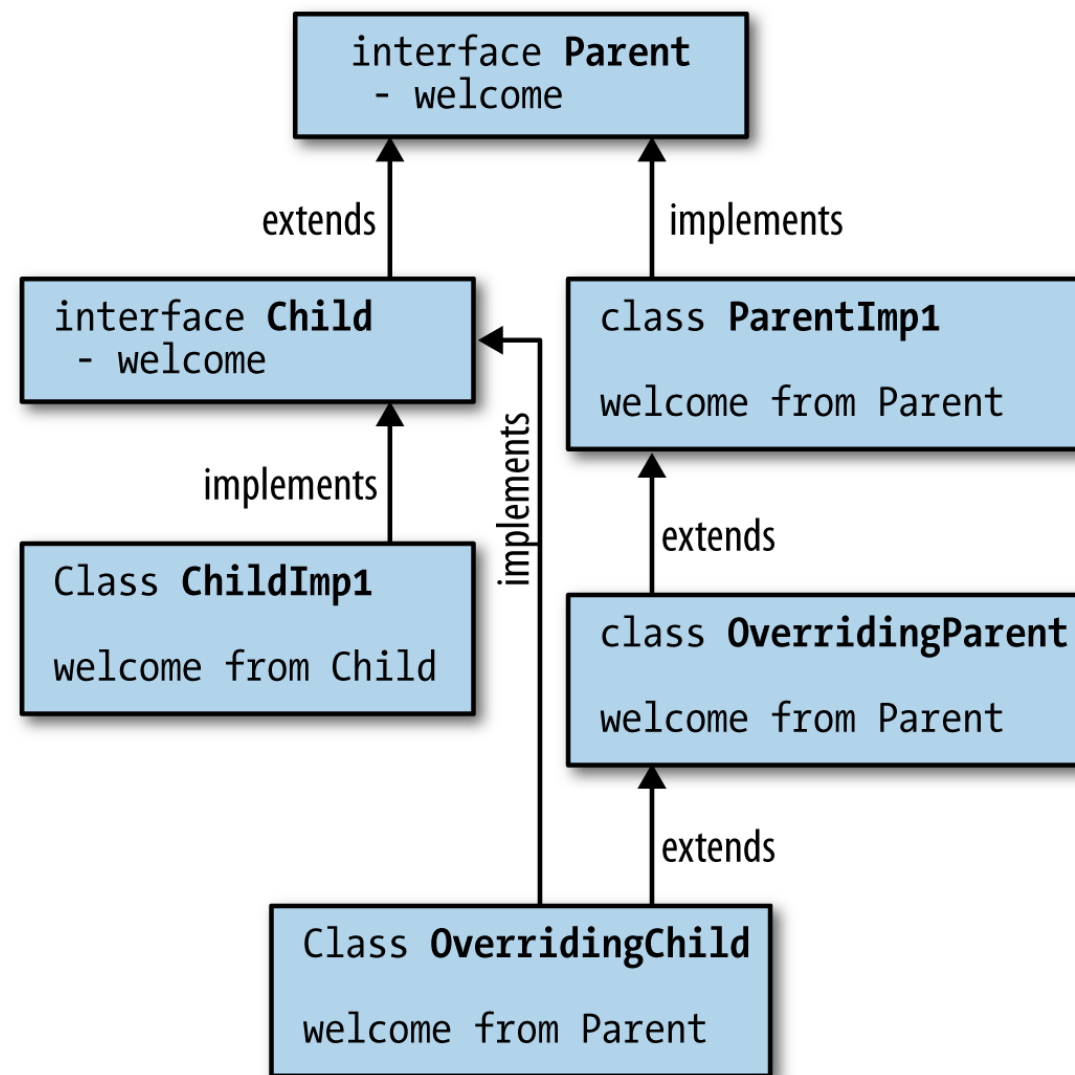
```
public interface Child extends Parent {
 @Override
 public default void welcome() { message("Child: Hi!"); }
} }
```

.





# A Deeper Situation



```
public class OverridingChild extends OverridingParent
implements Child { }
```

# Multiple Inheritance

- Interfaces are subject to multiple inheritance, it's possible to get into situations where two interfaces both provide default methods with the same signature.

```
public interface Jukebox {
 public default String rock() {
 return "... all over the world!"; }
}
```

```
public interface Carriage {
 public default String rock() { return "... from side to side";
} }
```

```
public class MusicalCarriage implements Carriage, Jukebox {
 @Override
 public String rock() {
 return Carriage.super.rock();
} }
```

# The Three Rules

1. Any class wins over any interface. So if there's a method with a body, or an abstract declaration, in the superclass chain, we can ignore the interfaces completely.
2. Subtype wins over super-type. If we have a situation in which two interfaces are competing to provide a default method and one interface extends the other, the subclass wins.
3. No rule 3. If the previous two rules don't give us the answer, the subclass must either implement the method or declare it abstract.

# Static Methods on Interfaces

- You may recall that Stream is an interface, but this is a static method on an interface. This is another new language change that has made its way into Java 8, primarily in order to help library developers, but with benefits for day-to-day application developers as well.

# Optional

- Optional is a new core library data type that is designed to provide a better alternative to null.
- null is often used to represent the absence of a value, and this is the use case that Optional is replacing.

```
Optional<String> a = Optional.of("a");
Optional emptyOptional = Optional.empty();
Optional alsoEmpty = Optional.ofNullable(null);
```

# Method References

artist -> artist.getName()

Artist::getName

(name, nationality) -> **new** Artist(name, nationality)

Artist::**new**

String[]::**new**

# Key Points

- A significant performance advantage can be had by using primitive specialized lambda expressions and streams such as `IntStream`.
- Default methods are methods with bodies on interfaces prefixed with the keyword `default`.
- The `Optional` class lets you avoid using `null` by modeling situations where a value may not be present.